Cahier des charges · octobre 2026

Tournoi d’échecs en direct

Partie en direct, spectateur et rejeu.
Java, Spring Boot, PostgreSQL, Kafka, Angular.

Statut	Cadrage validé. Développement pas encore commencé.
Auteur	Raouf Abdallah
Document	Référence du projet, du cadrage jusqu’à la mise en ligne.

Table des matières

1 Contexte **2**

2 Objectifs **2**

3 Utilisateurs et rôles **3**

4 Périmètre **3**

5 Parcours de démo **4**

6 Stack **4**

7 Architecture **5**

8 Modèle de données **5**

9 API **6**

10 Événements Kafka **6**

11 Règles côté serveur **6**

12 Frontend **7**

13 Qualité et tests **7**

14 Dépôt et construction **7**

15 Environnement local **8**

16 Intégration continue **8**

17 Déploiement **8**

18 Monitoring **8**

19 Sécurité **9**

20 Critères de fin **9**

21 Formulation CV **9**

1 Contexte

On joue souvent aux échecs entre amis. L'idée de départ est un petit tournoi le même soir, avec un suivi clair pour tout le monde.

Des plateformes existantes couvrent déjà ce besoin de jeu.

Plateforme	Ce qu'elle permet
Lichess	Tournoi privé gratuit : arène avec mot de passe, ou équipe puis Swiss / arène.
Chess.com	Shareable Tournament, lien privé, Swiss ou arène. En général au moins 4 joueurs pour une arène live.
Chessly	Apprentissage. Pas d'organisation de tournois.

Ce projet ne cherche pas à remplacer Lichess. Il sert à concevoir un produit temps réel, et à monter en compétence sur la stack la plus présente dans les offres junior en France : Java, Spring Boot, PostgreSQL, Docker, tests et Kafka.

Ce sujet plutôt qu'un CRUD, parce que :

— un seul coup est valide à la fois ;
— plusieurs écrans restent synchronisés : Blancs, Noirs, spectateur ;
— un coup illégal ou hors tour est refusé par le serveur ;
— la partie se rejoue depuis un journal d'événements.

2 Objectifs

2.1 Produit

— Deux amis jouent une partie en direct dans le navigateur.
— Un troisième écran suit la partie sans pouvoir jouer.
— Un coup illégal est refusé.
— La partie se rejoue coup par coup depuis l'historique.
— Plus tard : un mini-tournoi entre amis, avec invitation, parties et classement.

2.2 Technique

Ce que le CV et l'entretien doivent pouvoir défendre, une fois le projet livré.

— API Java 21 / Spring Boot 3, source de vérité côté serveur.
— PostgreSQL, migrations Flyway, unicité du numéro de coup.
— Spring Security et JWT, rôles joueur et spectateur.

— Kafka, avec outbox dans la même transaction que le coup.
— Second service qui consomme Kafka et pousse le direct.
— Docker Compose, JUnit, Mockito, Testcontainers, GitHub Actions.
— Front Angular et TypeScript qui affiche l’échiquier, sans décider si un coup est légal.

2.3 Hors objectifs

— Moteur d’analyse : Stockfish, évaluation, flèches.
— Classement Elo mondial, anti-triche, matchmaking public.
— Kubernetes, Spring Cloud, gRPC, Elasticsearch, Keycloak.
— Application mobile native.
— Remplacer Chess.com ou Lichess.

3 Utilisateurs et rôles

Rôle	Peut
Visiteur	Créer un compte, se connecter.
Joueur Blanc ou Noir	Rejoindre sa partie, jouer uniquement à son tour.
Spectateur	Ouvrir une partie avec son code, suivre les coups, lancer le rejeu.
Organisateur (V2)	Créer un tournoi, inviter des amis, voir le classement.

Des comptes de démo sont prévus pour l’entretien : deux joueurs et un spectateur. Les identifiants vivent dans le README, pas dans ce document.

4 Périmètre

4.1 V1, partie en direct

Obligatoire avant le tournoi.

— Inscription et connexion JWT.
— Créer une partie, obtenir un code, le second joueur rejoint.
— Échiquier, coup envoyé au serveur, diffusion aux deux joueurs.
— Refus : coup illégal, mauvais tour, spectateur, double envoi du même numéro.
— Fin de partie : mat, pat, abandon.
— Promotion du pion : le joueur choisit la pièce, le serveur l’applique.
— Spectateur en lecture seule, rejeu ordonné.
— Lancement local avec Docker Compose, README avec schéma et parcours de démo.

4.2 V2, tournoi entre amis

Uniquement quand la V1 est stable.

— Créer un tournoi, inviter par code.
— Rattacher les parties au tournoi.
— Appariement simple : Swiss léger ou arène à 4 joueurs. Pas de round robin aller-retour.
— Page de classement : points et parties jouées.

4.3 Reporté

— Contrôle du temps (blitz, pendule), après une V1 déjà solide.
— Chat, notifications e-mail.
— Plusieurs parties simultanées au-delà du besoin de démo.

5 Parcours de démo

Trois fenêtres côte à côte.

1. Les Blancs jouent un coup.
2. Les Noirs le voient apparaître.
3. Le spectateur le voit aussi, et ne peut pas jouer.
4. Un coup illégal est refusé : l'échiquier ne change pas.
5. Le bouton Rejouer reconstruit la partie coup par coup.

Si l'on demande pourquoi ce projet :

> On joue souvent entre amis. Organiser notre tournoi était l'occasion. Lichess et Chess.com savent déjà le faire gratuitement. L'objectif est de concevoir le produit : validation côté serveur, événements, persistance, spectateur et rejeu, sur une stack très présente dans les offres Java en France.

6 Stack

6.1 Backend

Élément	Rôle
Java 21, Spring Boot 3, Maven	Socle des deux services.
Spring Web	API REST.
Spring Data JPA, PostgreSQL, Flyway	Persistance et migrations.
Spring Security, JWT	Authentification et rôles.
Bean Validation	Entrées de l'API.
Apache Kafka	Faits de jeu, après l'écriture en base.
chesslib	Règles, FEN, coups légaux, mat et pat.

Deux services, pas plus en V1.

— `game-service` : comptes, parties, coups, persistance, publication.
— `live-service` : consommation Kafka, push temps réel, rejeu.

Le serveur n'accepte un coup que s'il figure dans les coups légaux renvoyés par chesslib.

6.2 Frontend

— Angular, TypeScript, Angular CLI.
— chessground pour l'échiquier, dans un composant Angular. Affichage et glisser-déposer uniquement. Il ne valide pas les règles.
— Composants, services, routeur et HttpClient pour le REST. WebSocket ou SSE pour le direct.

6.3 Outillage

Docker, Docker Compose, JUnit 5, Mockito, Testcontainers, GitHub Actions. Éditeur : Cursor, avec les extensions Java et Spring Boot.

OpenAPI et Redis restent optionnels. Redis n'entre que si la présence ou les sessions deviennent un vrai problème.

7 Architecture

Composant	Rôle
Navigateur	Angular et chessground. Propose un coup, affiche le FEN reçu.
game-service	Décide si le coup est légal. Écrit la partie, le coup et l'outbox.
PostgreSQL	Source de vérité : parties, coups, outbox.
Kafka	Topic `chess.game.events`, clé `gameId`.
live-service	Consomme les événements et pousse les trois écrans.

— Le coup, la mise à jour de la partie et la ligne d'outbox sont dans la même transaction.
— Le publisher envoie l'outbox vers Kafka après le commit. Si Kafka est indisponible, le coup reste en base et sera republié.
— `live-service` ne valide pas les règles. Il diffuse ce que `game-service` a déjà accepté.
— L'ordre par partie vient de `move_number` en base et de la clé Kafka.
— Pas de troisième service tant que ces deux rôles restent clairs.

8 Modèle de données

Migrations uniquement via Flyway. Pas de `ddl-auto=update` en dehors d'un essai local jetable.

8.1 users

`id`, `email` unique, `password_hash`, `display_name`, `created_at`.

8.2 games

— `id`, `code` unique, court, partageable.
— `white_user_id` et `black_user_id`, vides jusqu'au join.
— `status` : `WAITING`, `IN_PROGRESS`, `FINISHED`.
— `result` : `WHITE_WIN`, `BLACK_WIN`, `DRAW`, ou vide.
— `end_reason` : `CHECKMATE`, `STALEMATE`, `RESIGN`, ou vide.
— `current_fen`, `next_move_number` (démarre à 1), `created_at`, `updated_at`.

8.3 moves

`id`, `game_id`, `move_number`, `player_color` (`WHITE` ou `BLACK`), `uci` (`e2e4`, `e7e8q` pour une promotion), `san` (`e4`, `e8=Q`), `fen_after`, `created_at`.

Contrainte unique sur `(game_id, move_number)`.

8.4 outbox_events

`id`, `game_id`, `event_type`, `payload` JSON, `move_number`, `created_at`, `published_at`. Ce dernier reste vide tant que Kafka n'a pas acquitté.

8.5 V2 uniquement

`tournaments` (`id`, `name`, `code`, `status`, `created_by`), `tournament_participants` (`tournament_id`, `user_id`, `score`), et `games.tournament_id` nullable.

9 API

Authentification : `Authorization: Bearer <jwt>`.

Méthode	Chemin	Effet
POST	`/auth/register`	Crée un compte.
POST	`/auth/login`	Renvoie un JWT.
POST	`/games`	Crée une partie et son code. Blancs par défaut.
POST	`/games/{code}/join`	Le second joueur prend la couleur libre.
GET	`/games/{code}`	FEN, statut, joueurs, dernier coup.
POST	`/games/{code}/moves`	Joue un coup. Corps : `uci`.
POST	`/games/{code}/resign`	Abandon du joueur authentifié.
GET	`/games/{code}/moves`	Historique ordonné, pour le rejeu.

Le spectateur s'abonne au flux du `live-service`, par `gameId` ou par code, sans être assis à l'échiquier.

Codes d'erreur stables, en JSON :

— `NOT_YOUR_TURN`, `ILLEGAL_MOVE`, `SPECTATOR_CANNOT_MOVE`
— `GAME_NOT_STARTED`, `GAME_ALREADY_FINISHED`, `GAME_FULL`

Le flux pousse, après `move.played` et `game.ended`, le FEN, le coup, le numéro et le statut. CORS : origine du front local, puis l'URL de production.

10 Événements Kafka

Topic `chess.game.events`. Clé `gameId`. Un type par message.

Type	Contenu
`move.played`	gameId, moveNumber, uci, san, fenAfter, playedBy.
`game.ended`	gameId, result, endReason, finalFen.
`player.joined`	gameId, color, userId.

Le consumer tolère un message déjà vu (rejeu d'offset, republication de l'outbox). Appliquer un coup déjà connu ne duplique pas l'état.

11 Règles côté serveur

À chaque `POST /moves` :

1. Charger la partie et la verrouiller pour la transaction.
2. Vérifier le statut `IN_PROGRESS`, l'identité du joueur, et la couleur attendue.
3. Charger le FEN dans chesslib.
4. N'accepter le coup que s'il est dans `legalMoves()`. Ne pas se fier à `isMoveLegal` seul. Test obligatoire : un pion qui prend droit devant est refusé.

5. Appliquer le coup, calculer le FEN suivant, détecter mat et pat.
6. Insérer `moves` avec le numéro courant. La contrainte unique protège deux requêtes simultanées.
7. Mettre à jour `games` et insérer `outbox_events`.
8. Commit. La publication Kafka se fait après, par le publisher.

Promotion : l'UCI contient la pièce, par exemple `e7e8q`. Sans pièce de promotion sur la dernière rangée, la réponse est `ILLEGAL_MOVE` et le front redemande le choix.

12 Frontend

Écrans de la V1 :

— Connexion et inscription.
— Accueil : créer une partie, rejoindre avec un code, suivre avec un code.
— Partie : composant chessground, liste des coups, statut (à vous, à l'adversaire, terminée).
— Spectateur : même échiquier, aucun envoi de coup, badge spectateur.
— Rejeu : précédent / suivant, à partir de `GET /moves`.

Le front applique le FEN renvoyé par le serveur. Il peut prévisualiser les cases d'arrivée. Un refus serveur annule le coup visuel.

13 Qualité et tests

La V1 est présentable quand ces parcours sont couverts. Pas de couverture à 100 %.

— Unitaire : coup illégal, mauvais tour, spectateur, partie déjà finie, promotion.
— Concurrence : deux `POST /moves` en parallèle. Un seul gagne, un seul `move_number`.
— Intégration Testcontainers (PostgreSQL et Kafka) : trois coups produisent trois événements dans l'ordre, et le consumer reconstruit le même FEN que la base.
— Le README explique comment lancer les tests.

14 Dépôt et construction

```
tournoi-echecs/
CAHIER_DES_CHARGES.tex
README.md
docker-compose.yml
game-service/
live-service/
frontend/
.github/workflows/ci.yml
```

L'infrastructure locale est en place : PostgreSQL, Kafka et le topic `chess.game.events`. Les applications ne sont pas encore générées. L'ordre de construction ne s'inverse pas.

1. Infrastructure : Docker Compose, PostgreSQL, Kafka, topic.
2. Coup légal en REST et PostgreSQL.
3. Front Angular qui joue.
4. Outbox branché sur le Kafka déjà lancé.

5. `live-service`, spectateur, rejeu.
6. Tests d'intégration et CI.
7. Tournoi V2.
8. Mise en ligne.
9. Monitoring.

Durée indicative : 3 à 4 semaines à mi-temps, ou 2 semaines en continu, une fois Java et Spring en main.

15 Environnement local

Prérequis : JDK 21, Maven, Node.js LTS, Docker Desktop, Git.

`docker compose up` lance aujourd'hui PostgreSQL et Kafka. `game-service` et `live-service` rejoindront le même Compose quand ils existeront. Le front tournera avec `ng serve` et un proxy vers les API locales.

Les comptes de démo sont dans le README. Aucun secret réel dans Git.

16 Intégration continue

GitHub Actions, à chaque push :

— `mvn test` sur les deux services. Testcontainers a besoin de Docker sur le runner.
— `npm ci`, puis `ng build` du front Angular.
— Build des images Docker, sans publication tant que le déploiement n'existe pas.

17 Déploiement

Cible de démo : un VPS (Hetzner, Scaleway ou OVH, environ 5 à 15 euros par mois) avec Docker Compose. Le front statique est sur Vercel ou Cloudflare Pages.

Sur le VPS :

— Compose de production : PostgreSQL avec volume, Kafka, les deux services.
— Reverse proxy Caddy ou Nginx, HTTPS Let's Encrypt.
— PostgreSQL et Kafka non exposés sur Internet.
— Secret JWT et mots de passe uniquement en variables d'environnement.
— CORS limité à l'URL du front. WebSocket ou SSE autorisés par le proxy.

Une fois la V1 stable, GitHub Actions construit les images puis déploie par SSH. Pas de Kubernetes.

Un nom de domaine est optionnel. Une IP ou un sous-domaine suffit pour l'entretien. Un domaine est plus propre pour le CV.

18 Monitoring

Niveau adapté à une démo, pas à un système d'information bancaire.

— Spring Actuator sur les deux services : santé du processus, de PostgreSQL et de Kafka.
— Le reverse proxy ne publie pas les endpoints sensibles (environnement, heap dump).
— Journaux avec `gameId` et `moveNumber`. Info sur coup accepté, coup refusé, fin de partie, échec

de publication.
— Pas de mot de passe, pas de JWT complet dans les logs.
— Métriques : coups acceptés et refusés, lag du consumer, taille de l'outbox non publiée.

Sur le VPS : `docker compose logs`, healthchecks Compose, et une vérification manuelle avant une démo.

Sauvegarde : dump PostgreSQL quotidien tant que des parties comptent. Kafka n'est pas la source de vérité. La base et l'outbox le sont. Si le topic est perdu, on republie l'outbox.

Incident à savoir expliquer : Kafka tombe après le commit. Le coup est en base, `published_at` reste vide, le publisher reprend, le spectateur rattrape. Le joueur a déjà sa réponse HTTP.

19 Sécurité

Niveau projet personnel.

— Mots de passe hashés avec BCrypt.
— JWT signé, expiration courte, secret hors de Git.
— Un joueur ne joue que sa couleur. Le spectateur n'a pas de route de coup.
— Validation des entrées : UCI, longueur du code partie.
— Fichier `.env` ignoré par Git.

Hors V1 : OAuth, double authentification, chiffrement du trafic interne au-delà du réseau Docker.

20 Critères de fin

La V1 est terminée quand :

— la démo des trois écrans marche sur une machine propre, avec le README ;
— le coup illégal et le double envoi sont couverts par des tests ;
— `docker compose up` suffit pour l'API, la base et Kafka ;
— le README explique l'architecture en un schéma ;
— une URL publique permet de refaire la démo sans le PC du développeur.

La V2 est terminée quand quatre comptes jouent un mini-tournoi et qu'un classement s'affiche.

21 Formulation CV

À n'écrire qu'une fois la démo réelle.

— Partie d'échecs en direct : API Java 21 / Spring Boot, validation des coups côté serveur, front Angular.
— Journal des coups publié sur Kafka via un outbox, ordre garanti par un numéro de coup.
— Spectateur et rejeu reconstruits depuis les événements, tests Testcontainers sur les coups illégaux.

Java, Spring et Kafka n'entrent dans les compétences maîtrisées qu'à ce moment-là. Le projet reste dans une section Projets, pas dans les expériences professionnelles.